

Can a model learn to find what it needs?

A recursive language model (RLM) can use tools and ask helper models to solve parts of a task.

For this study, we returned to a basic skill.

Can better examples and better tools help a small model finish a multi-step task?

Example: Make a wooden pickaxe in a crafting game.

1. The model asks which ingredients it needs.
2. It makes the parts and then the pickaxe.
3. The game checks whether the pickaxe actually exists.

Simplified illustration of TextCraft, our test environment. These experiments use one model with tools, not recursive helper calls.

We taught the model by showing what to do next.

We trained copies of the same model to imitate worked examples. This is **supervised fine-tuning (SFT)**.

Example: Given “Make a pickaxe” and the actions so far, learn to ask for the pickaxe recipe next.

The teacher knows recipes.

A program plans the steps using recipes it already knows, then shows lookups and actions.

The teacher discovers recipes.

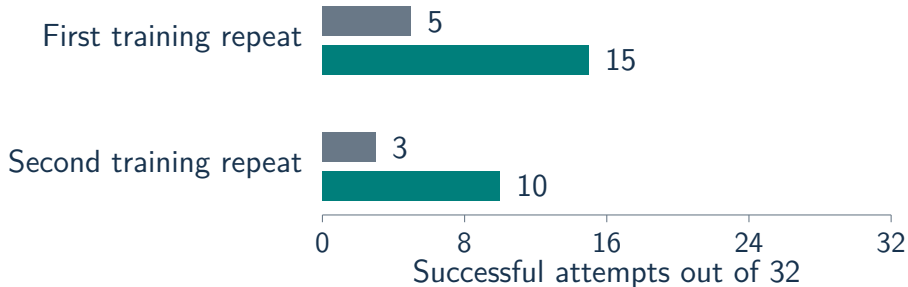
A program looks up the goal's recipe, discovers the needed parts, then looks up their recipes.

Both show lookups. Only the second uses them to decide the route.

Simplified example. Both sets cover the same 32 training tasks and number of steps, but differ in text and action order.

Learning from discovery led to more finished tasks.

We repeated training twice. Each time, we tested both models on the same 16 tasks, with two attempts per task.



■ Teacher knows recipes ■ Teacher discovers recipes

All outcomes are known. We corrected an ingredient-quantity error in the first teacher's examples before this comparison. These are exploratory tests, not a final untouched test set.

The advantage remained when recipes changed.

Could the models cope when making the same items required different ingredients? We changed the recipes and starting supplies, without retraining.

Successful attempts (out of 16)	First repeat	Second repeat
Teacher knows the recipes	4	1
Teacher discovers the recipes	6	8

The lead was small in one repeat and large in the other.

This is encouraging, but it is still **one kind of game**.

Eight goals, two attempts each; all outcomes are known. The quantity error is corrected here too. The smaller difference is too uncertain to establish a clear advantage on its own.

Let the model choose the step; let code fill the details.

The model chooses what to make and how much. Code fills in the ingredients using a recipe the model already looked up.

We changed the tool, with **no extra model training**.

Successful attempts (out of 16)	Model fills ingredients	Code fills ingredients
Slide 4 recipes, first model	6	12
Slide 4 recipes, second model	8	10
Different recipes, first model	9	14

Next question: Can this division of work help on other tasks?

Both models learned from discovery examples. These are small, related tests in one game; the smallest gain remains uncertain. We have not shown better recursive planning.